

ВВЕДЕНИЕ

Индустрия пляжного отдыха, являющаяся важной составляющей туристической экономики многих регионов, сегодня сталкивается с острой потребностью в цифровой трансформации процессов бронирования и управления инфраструктурой. Отсутствие единого цифрового пространства приводит к тому, что отдыхающие не могут гарантированно забронировать шезлонги или заранее заказать услуги, а владельцы пляжного бизнеса лишены инструментов для автоматизации учета, управления спросом и сбора статистики. Существующие разрозненные решения не интегрированы между собой и не обеспечивают централизованного хранения данных о доступности услуг, загрузке пляжа и предпочтениях клиентов. Это создает технологический разрыв между растущими ожиданиями пользователей и реальными возможностями пляжной инфраструктуры.

С технической точки зрения, создание эффективной платформы агрегации пляжных услуг требует разработки надежной, масштабируемой и безопасной серверной инфраструктуры. Ключевыми задачами становятся проектирование базы данных, способной хранить и обрабатывать информацию о тысячах объектов (пляжи, точки проката, заведения), реализация бизнес-логики бронирования с контролем транзакций и предотвращением двойных записей, а также создание защищенных API для взаимодействия с клиентскими приложениями. Особую сложность представляет необходимость обработки пиковых нагрузок в курортный сезон, обеспечение согласованности данных при параллельных запросах и интеграция с внешними платежными сервисами. Таким образом, разработка стартапа представляет собой комплексную задачу в области серверной разработки, проектирования баз данных и построения отказоустойчивых распределенных систем.

Объектом исследования в данной выпускной квалификационной работе являются процессы обработки данных, управления бронированиями и

взаимодействия между участниками пляжного сервиса (отдыхающими, поставщиками услуг, администраторами).

Предметом исследования выступают серверные архитектуры, модели данных, алгоритмы обработки транзакций, методы обеспечения целостности и безопасности данных, а также способы организации взаимодействия между компонентами распределенной системы (API, базы данных, внешние сервисы).

Целью выпускной квалификационной работы является разработка и реализация функциональной платформы сервиса агрегации услуг и управления пляжной инфраструктурой для удобного взаимодействия пользователя с представителями малого бизнеса.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

- провести технический анализ существующих решений в сфере управления пляжной инфраструктурой с точки зрения их серверной архитектуры, моделей данных и используемых протоколов взаимодействия;

- спроектировать серверную архитектуру платформы, разработать логическую и физическую модели базы данных, определить состав микросервисов (или монолитной структуры) и выбрать технологический стек;

- разработать и реализовать серверную логику обработки ключевых бизнес-процессов: регистрация и аутентификация пользователей, управление каталогом услуг, обработка бронирований, проведение платежей, формирование отчетности;

- создать RESTful API для взаимодействия с клиентской частью, обеспечив документирование эндпоинтов, обработку ошибок и валидацию входящих данных;

- обеспечить интеграцию с внешними сервисами: платежными системами, геоинформационными сервисами (карты), SMS- и email-рассылками;

провести тестирование серверной части (модульное, интеграционное, нагрузочное), оптимизировать производительность запросов к базе данных и обеспечить безопасность серверных компонентов.

В работе применяется комплекс методов исследования: системный анализ предметной области, методы проектирования реляционных и нереляционных баз данных (нормализация, индексирование, оптимизация запросов), методы объектно-ориентированного проектирования серверной логики, алгоритмические методы обработки транзакций и управления конкурентным доступом, а также экспериментальные методы нагрузочного тестирования серверной части для оценки производительности и масштабируемости разработанных решений. Информационную базу составят техническая документация по используемым серверным технологиям, стандарты безопасности при обработке платежей, а также результаты тестирования разработанных серверных модулей.

Практическая значимость работы заключается в возможности использования разработанной серверной архитектуры, базы данных и API в деятельности операторов пляжной инфраструктуры, администраций курортных зон и агрегаторов туристических услуг. Реализованные бэкенд-решения могут быть развернуты как самостоятельная платформа для управления пляжными территориями или интегрированы в существующие туристические экосистемы, обеспечивая надежное хранение данных, обработку транзакций и масштабирование при росте числа пользователей и подключаемых объектов инфраструктуры.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Теоретико-аналитическая часть работы направлена на обоснование подходов к проектированию серверной части цифровой платформы для агрегации услуг пляжной инфраструктуры. В центре анализа находится серверная архитектура системы: доменная модель, хранение и обработка данных, API-контракты, правила доступа, транзакционная согласованность и интеграции с внешними сервисами. Такая постановка соответствует общим принципам проектирования информационных систем, в которых архитектурное решение рассматривается как результат анализа предметной области и требований к качеству программного продукта [1].

Необходимость отдельного анализа обусловлена особенностями предметной области. В пляжных зонах сервис предоставляется множеством независимых операторов: точками питания, пунктами проката, серф-станциями, организаторами водных активностей и арендаторами инвентаря. Для пользователя это означает разрозненные каналы взаимодействия, а для бизнеса - отсутствие единого цифрового контура, в котором можно управлять услугами, доступностью ресурсов, заказами и аналитикой.

С позиции backend-разработчика сложность проекта состоит в координации нескольких взаимосвязанных процессов: регистрации и аутентификации пользователей, управления каталогом услуг, резервирования мест и временных слотов, обработки заказов, платежных операций, отправки уведомлений и разграничения прав доступа. Следовательно, задача заключается не в создании набора отдельных форм, а в построении серверной системы, которая обеспечивает целостность данных и корректное выполнение бизнес-процессов при изменении состояния объектов во времени.

Цель главы заключается в теоретическом обосновании архитектурных и методических решений для серверной части платформы. Для достижения этой цели необходимо раскрыть предметную область, сформулировать общую проблему и серверную задачу, сопоставить существующие решения, сравнить

архитектурные подходы, определить возможные пути решения и описать методику дальнейшего проектирования.

1.1 Анализ предметной области и формулировка проблемы

Предметная область проекта связана с цифровизацией сервисов отдыха и досуга в пляжных зонах. На практике пользователь взаимодействует с несколькими разрозненными источниками информации: мессенджерами, социальными сетями, сайтами отдельных точек и офлайн-кассами. В результате поиск свободного места, аренда инвентаря, заказ еды или запись на активность требуют избыточного числа действий и не гарантируют актуальности данных.

С серверной точки зрения данная проблема обусловлена не только отсутствием удобного интерфейса, но прежде всего недостаточной связанностью данных и процессов. Если сведения об организациях, расписании, доступности ресурсов, статусах заказов и правах сотрудников не собраны в единой системе, пользователь не может получить достоверный сценарий обслуживания, а партнер - управляемый контур учета. Поэтому проблема предметной области переходит в задачу проектирования централизованной серверной модели, объединяющей участников рынка и жизненный цикл услуг.

В проекте выделяются три основные группы участников: клиент, организация-партнер и администратор платформы. Для клиента значимы поиск, фильтрация, бронирование и оплата. Для партнера - управление карточкой организации, каталогом услуг, расписанием, схемой размещения, доступностью ресурсов и заказами. Для администратора - модерация, контроль контента, управление справочниками и разрешение спорных ситуаций. Такая структура требует формализованной ролевой модели доступа, основанной на разграничении прав по ролям и операциям, что соответствует принципам RBAC-моделей [2].

Дополнительная сложность определяется природой самих услуг. Часть сервисов имеет пространственную привязку, например аренда лежака, беседки или столика на схеме объекта. Другая часть описывается временным слотом, количеством ресурсов или статусом заказа, например прокат оборудования, экскурсия, доставка из кафе. Следовательно, модель данных должна быть достаточно гибкой, чтобы единообразно представлять как объекты со схемой, так и услуги без пространственной разметки.

Еще одной особенностью предметной области является высокая чувствительность к изменению состояния данных во времени. Место может быть свободно, временно зарезервировано, оплачено, отменено или освобождено. Аналогично заказ проходит последовательность состояний от создания до завершения. Для серверной части это означает необходимость формализованной модели состояний, проверки допустимости переходов и контроля конкурентных операций.

Таким образом, общая проблема предметной области состоит в фрагментированности цифрового обслуживания в сфере пляжной инфраструктуры. В серверной проекции она выражается как отсутствие единой архитектуры данных и бизнес-логики, способной поддерживать бронирование, управление услугами, заказы, роли и интеграции в рамках одной информационной системы.

1.2 Анализ существующих решений и их ограничений с позиции серверной архитектуры

Для выбора архитектурного подхода необходимо сопоставить реальные цифровые решения, которые частично закрывают задачи исследуемой предметной области. В качестве ориентиров выбраны российские сервисы Azur.ru [3], MAYAK.TRAVEL [4] и TVIL.RU [5]. Первый представляет собой информационный каталог курортного отдыха с прямыми контактами объектов размещения, второй объединяет каталог мест, событий и услуг с возможностью оформления отдельных заказов, третий реализует модель

онлайн-бронирования объектов размещения с календарем занятости, комиссионной моделью и инструментами для партнеров. Ни одно из этих решений не ориентировано непосредственно на управление пляжной инфраструктурой, однако они позволяют выявить сильные и слабые стороны существующих backend-моделей.

Сравнение выполняется по единым backend-критериям: полнота доменной модели, наличие централизованного хранения данных, поддержка ролевого доступа, интеграционные возможности и пригодность к масштабированию. Такой способ сравнения позволяет оценивать не визуальную оболочку сервисов, а архитектурную логику обработки данных и сервисных взаимодействий. Результаты сопоставления представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ существующих типов решений с позиции серверной реализации

Тип решения	Что закрывает	Роли и доступ	Интеграции	Ключевое ограничение
Информационный каталог с прямыми контактами (Azur.ru [3])	Описание курортов, пляжей и объектов размещения, публикация контактов и ориентировочных цен	Нет выраженной ролевой модели	Минимальные	Нет единого транзакционного контура бронирования и заказа, данные носят справочный характер
Мультисервисный туристический каталог (MAYAK.TRAVEL [4])	Каталог мест, событий, услуг, ресторанов и отдельных заказов	Разделение пользовательских сценариев	Есть	Широкий туристический профиль, но отсутствует серверная модель управления пляжными ресурсами и локальными операторами
ОТА-сервис бронирования размещения (TVIL.RU [5])	Онлайн-бронирование объектов размещения, календарь занятости, комиссия,	Развита на собственном домене	Есть	Ориентация на размещение и календарную доступность, а не на смешанную модель мест,

	инструменты для партнеров			услуг и пляжной инфраструктуры
--	------------------------------	--	--	-----------------------------------

Данные таблицы 1.1 показывают, что рассмотренные решения закрывают разные фрагменты пользовательского пути. Azur.ru сильнее как информационный и справочный ресурс, но не формирует единого транзакционного контура заказа. MAYAK.TRAVEL ближе к мультисервисной модели, однако ориентирован на широкий туристический каталог и не решает задачу управления ресурсами пляжной инфраструктуры. TVIL.RU обеспечивает зрелый сценарий онлайн-бронирования, но в первую очередь работает с объектами размещения и календарной доступностью, а не с комбинированной моделью мест, услуг, доставки и локальных операторов.

Из сопоставления следует, что для исследуемой платформы требуется серверная модель, которая одновременно поддерживает каталог организаций, схематизированное бронирование ресурсов, дополнительные услуги, роли сотрудников, уведомления и интеграционные сценарии. Иначе говоря, задача проекта заключается не в копировании одного из готовых сервисов, а в объединении функций, которые в существующих решениях распределены между разными платформами.

Таким образом, общая задача, решаемая в данном исследовании, может быть сформулирована следующим образом: необходимо спроектировать серверную архитектуру платформы агрегации услуг пляжной инфраструктуры, которая обеспечивает централизованное хранение и согласованную обработку данных о пользователях, организациях, услугах, бронированиях и заказах, поддерживает ролевое разграничение доступа, интеграцию с внешними сервисами и дальнейшее масштабирование решения.

1.3 Выбор архитектурного подхода и возможных путей решения

Определим архитектурный подход, который может быть положен в основу серверной части платформы. Для рассматриваемого проекта теоретически допустимы три базовых варианта: монолитная архитектура,

модульный монолит и микросервисная архитектура. Их сопоставление важно проводить именно с позиции backend-задач, поскольку выбор архитектуры напрямую влияет на модель данных, способы интеграции, сопровождение и масштабирование системы.

Монолитная архитектура предполагает единое серверное приложение, общую базу данных и общий цикл развертывания. Ее сильная сторона - быстрый старт и низкие инфраструктурные издержки. Однако при росте числа доменов такая архитектура усиливает связанность компонентов, усложняет независимое изменение бизнес-логики и делает рискованными крупные релизы. Для платформы с каталогом организаций, бронированием, заказами, уведомлениями и административным контуром этот вариант оправдан только при крайне ограниченном объеме функциональности [1].

Модульный монолит является более сдержанным и практически оправданным вариантом для раннего этапа. Он позволяет сохранить единый процесс развертывания, но при этом структурировать код по предметным модулям. Такой подход может быть рационален, если команда стремится быстрее собрать минимально жизнеспособную версию продукта и может дисциплинированно выдерживать внутренние границы модулей. Ограничение состоит в том, что при росте числа интеграций и различии нагрузочных профилей система все равно остается связанной на уровне релиза, инфраструктуры и, как правило, общей модели данных [1; 6].

Микросервисная архитектура строится на разделении системы на самостоятельные сервисы, каждый из которых отвечает за свой домен и владеет собственной моделью данных. Для рассматриваемого проекта такой подход целесообразен не сам по себе, а потому, что контуры аутентификации, организаций, бронирования, заказов, уведомлений и медиа различаются по логике, частоте изменений и интеграционным требованиям. В литературе по микросервисной архитектуре именно независимое развитие сервисов, явные границы доменов и отдельные интерфейсы рассматриваются как признаки подхода [6; 7].

Сравнение архитектур проводится по пяти критериям: скорости старта, степени связанности компонентов, удобству сопровождения, масштабируемости и пригодности к доменной декомпозиции. Такой набор критериев соответствует подходам, применяемым в литературе по программной архитектуре и проектированию распределенных систем [1; 6; 7].

Таблица 1.2 – Сравнение архитектурных подходов для серверной части платформы

Подход	Преимущества	Недостатки	Пригодность для проекта	Вывод
Монолит [1]	Быстрый старт, простое развертывание, единая транзакционная модель	Сильная связанность компонентов, сложность масштабирования, затрудненное развитие доменов	Подходит только для очень ограниченного объема функций	Допустим как краткосрочный стартовый вариант
Модульный монолит [1; 6]	Лучшая структурированность кода, единое окружение, контролируемый рост	Общий цикл релиза, единая инфраструктурная точка отказа, высокая связность при росте интеграций	Приемлем для раннего этапа и дисциплинированной декомпозиции	Рационален как промежуточная модель
Микросервисная архитектура [6; 7]	Изоляция доменов, независимое развитие сервисов, гибкие интеграции, масштабируемость	Сложнее инфраструктура, выше требования к мониторингу и API-контрактам	Лучше соответствует структуре предметной области	Предпочтительна для целевого состояния платформы

Из таблицы 1.2 следует, что микросервисная архитектура не является единственно возможным решением и не должна рассматриваться как универсальный выбор для любого проекта. Для раннего этапа разработки модульный монолит остается технически допустимым вариантом. Тем не менее для целевого состояния платформы микросервисная модель лучше согласуется с предметной областью, поскольку позволяет развивать домены бронирования, заказов, уведомлений и интеграций без постоянного усложнения общего серверного ядра.

При этом выбор целевой микросервисной архитектуры не исключает поэтапную реализацию. На старте отдельные домены могут разворачиваться ограниченным составом сервисов, а часть вторичных функций может оставаться внутри укрупненных модулей. Такой переходный вариант позволяет не усложнять инфраструктуру преждевременно, но сохраняет корректные архитектурные границы.

Следовательно, в работе в качестве целевой принимается микросервисная архитектура, тогда как модульный монолит рассматривается как допустимый промежуточный вариант для раннего этапа внедрения, если требуется ускорить сборку минимально жизнеспособного продукта без отказа от доменной декомпозиции.

Роль backend-разработчика в проекте заключается в построении серверной основы цифровой платформы: проектировании логической и физической моделей данных, разработке API, реализации бизнес-логики, разграничении прав доступа, организации интеграций и обеспечении нефункциональных характеристик системы.

Серверное решение должно поддерживать несколько взаимосвязанных контуров: пользователей и роли, организации и точки обслуживания, каталог услуг, бронирование, заказы сопутствующих сервисов, уведомления, хранение медиа и административное управление. Для каждого из этих контуров необходимы собственные сущности, правила доступа и согласованная модель изменений состояния данных.

Существенным требованием является обеспечение ролевой модели доступа и интеграции с внешними сервисами: платежными шлюзами, картографическими сервисами и каналами уведомлений. При этом серверная часть должна оставаться расширяемой, чтобы новые типы услуг и партнерских сценариев можно было добавлять без переработки всей системы.

Для внутренней организации серверных компонентов целесообразно использовать принципы гексагональной архитектуры. Такой подход позволяет отделить доменную логику от инфраструктурного слоя, что упрощает

тестирование, замену адаптеров и сопровождение кода при развитии платформы.

Проектируемая платформа обладает рядом особенностей, которые непосредственно влияют на серверную часть. Первая особенность – смешанная природа услуг: система должна одинаково корректно работать и с объектами, привязанными к схеме мест, и с услугами без пространственной привязки. Вторая особенность – сочетание бронирования, сопутствующих заказов, уведомлений, медиа и модерации в рамках одного продукта. Третья особенность – многоуровневая ролевая модель, поскольку в системе одновременно действуют клиент, партнер и администратор.

Перечисленные особенности определяют требования к серверному решению: оно должно поддерживать событийную обработку состояний, доменную изоляцию сервисов, согласованность данных и возможность поэтапного развития архитектуры без потери управляемости.

1.5 Методические основания проектирования серверной части

Методическое основание проектирования серверной части строится на последовательном переходе от анализа предметной области к архитектурной декомпозиции и затем к выбору средств реализации. На практическом уровне это означает, что сначала выделяются устойчивые домены и ключевые сценарии обработки данных, затем определяется архитектурная форма их взаимодействия, после чего уточняются модель хранения, API-контракты и интеграционные точки.

Сравнение возможных вариантов реализации проводится по четырем признакам: полнота автоматизации, техническая управляемость, способность к расширению и эксплуатационная сложность. Такой способ сопоставления позволяет связать выбор архитектуры не с предпочтением конкретной технологии, а с тем, насколько она соответствует структуре предметной области и условиям развития платформы.

Таблица 1.3 – Сравнение возможных путей решения серверной задачи

Вариант	Сущность подхода	Сильные стороны	Слабые стороны	Оценка применимости
Минимальный backend	Часть логики переносится в ручные процессы	Быстрый старт	Низкая надежность, слабая масштабируемость	Подходит только как временная схема
Модульный монолит	Все домены реализуются в едином приложении с внутренними модулями	Умеренная сложность, понятная структура	Рост связанности при расширении функционала	Приемлемо для ограниченного старта
Поэтапный переход к микросервисной архитектуре	Доменная декомпозиция с постепенным выделением сервисов	Масштабируемость, изоляция ответственности, гибкие интеграции	Выше сложность проектирования и сопровождения	Оптимально для целевого развития платформы

Как видно из таблицы 1.3, для долгосрочного развития платформы наилучшим вариантом является поэтапное построение микросервисной архитектуры. При этом модульный монолит может рассматриваться как допустимый промежуточный вариант для ранних этапов, однако в целевом состоянии он уступает микросервисному подходу по степени доменной изоляции и удобству дальнейшего масштабирования.

Практическое проектирование серверной части в рамках выбранного подхода включает определение состава сервисов, формирование модели данных, описание API-взаимодействий, выбор технологического стека, выбор интеграционных механизмов и постановку требований к тестированию, наблюдаемости и безопасности.

В качестве базового технологического стека для серверной части платформы выбраны .NET, PostgreSQL, REST API и MinIO как S3-совместимое объектное хранилище. Выбор .NET обусловлен тем, что данная платформа обеспечивает высокую производительность серверных приложений, поддерживает строгую типизацию, удобную организацию многослойной архитектуры и имеет развитую экосистему для построения микросервисов и веб-сервисов. PostgreSQL выбран как основное хранилище, поскольку он поддерживает надежную транзакционную модель, реляционные

связи между сущностями, индексацию и работу со сложными структурами данных, что важно для сценариев бронирования, заказов, ролей и журналирования изменений. REST API выбран в качестве базового механизма взаимодействия между сервисами и клиентскими приложениями, так как он обеспечивает понятные контрактные интерфейсы, хорошо сочетается с HTTP-инфраструктурой и позволяет поэтапно развивать распределенную серверную систему без усложнения интеграционного контура. MinIO используется как S3-совместимое объектное хранилище для медиафайлов, поскольку такой подход позволяет отделить бинарные объекты от основной транзакционной базы данных, упростить масштабирование хранения и обеспечить унифицированный доступ к пользовательским изображениям и иным медиа-ресурсам.

Такой методический подход позволяет связать архитектурные решения с особенностями предметной области и обеспечить последовательный переход к практической реализации серверной части платформы.

1.6 Выводы по главе

Проведенный теоретико-аналитический анализ показывает, что ключевая проблема предметной области состоит в разрозненности цифрового обслуживания в сфере пляжной инфраструктуры и отсутствии единой серверной среды, объединяющей клиентов, партнеров и администрацию платформы. Данная проблема не устраняется только интерфейсными средствами, поскольку ее источник находится на уровне архитектуры данных, бизнес-логики и интеграционного взаимодействия.

Сравнение существующих решений показывает, что Azur.ru, MAYAK.TRAVEL и TVIL.RU закрывают отдельные сценарии цифрового обслуживания, однако не формируют единую серверную модель для координации пляжной инфраструктуры, сопутствующих сервисов и ролевого взаимодействия участников.

Таким образом, теоретико-аналитическая часть создает основу для последующего проектирования серверной архитектуры, модели базы данных, API-контрактов и интеграционного контура платформы.

2. РАЗРАБОТКА ПРОЕКТА

2.1 Концепция проекта

Общей целью проекта является создание веб-платформы для агрегации услуг и управления пляжной инфраструктурой, ориентированной на муниципальные пляжи с множеством независимых операторов. Платформа призвана решить проблему фрагментированности пляжного сервиса, объединив в едином цифровом пространстве малый бизнес (кафе, пункты проката инвентаря, серф-станции, организаторов водных развлечений) и отдыхающих, заинтересованных в удобном планировании своего досуга. Ключевая идея заключается в создании бесшовного пользовательского опыта, позволяющего туристам находить, бронировать и оплачивать пляжные услуги в несколько кликов, а представителям бизнеса - получать эффективные инструменты для управления спросом, динамического ценообразования и сбора аналитики.

2.2 Целевые ориентиры участников команды

В рамках достижения общей цели каждый участник команды фокусируется на решении специфических задач, соответствующих его профессиональной роли.

Для UI/UX-дизайнера целью является проектирование интуитивно понятного, эстетически привлекательного и адаптивного интерфейса, обеспечивающего комфортное взаимодействие с платформой как для отдыхающих, так и для представителей бизнеса. Дизайнер стремится создать визуально целостный продукт, в котором пользовательские сценарии будут проработаны до уровня, исключающего когнитивную перегрузку и снижающего порог входа для всех категорий пользователей.

Для фронтенд-разработчика целью является реализация клиентской части платформы, обеспечивающей высокую производительность, адаптивность и корректное отображение на всех типах устройств. Фронтенд-

разработчик отвечает за воплощение дизайн-макетов в работающий код, создание интерактивных компонентов (интерактивная карта пляжа, календарь бронирования, система фильтрации) и надежную интеграцию с серверными API для обеспечения плавного и отзывчивого пользовательского опыта.

Для бэкенд-разработчика целью является проектирование и реализация надежной, масштабируемой и безопасной серверной инфраструктуры. Бэкенд-разработчик создает базу данных, разрабатывает API для взаимодействия с клиентской частью, реализует бизнес-логику обработки бронирований и платежей, а также обеспечивает интеграцию с внешними сервисами (платежные системы, геоинформационные сервисы, каналы уведомлений), гарантируя стабильность работы платформы даже в условиях пиковых нагрузок.

Для достижения поставленной цели необходимо решить следующий комплекс задач:

1. Провести анализ предметной области, существующих платформ-конкурентов и целевой аудитории для выявления ключевых потребностей пользователей и формирования требований к функционалу.

2. Спроектировать информационную архитектуру платформы, разработать пользовательские сценарии и создать прототипы интерфейса низкой и высокой детализации, учитывающие потребности двух типов пользователей (отдыхающие и администраторы/бизнес).

3. Разработать клиентскую архитектуру приложения, выбрать технологический стек и реализовать пользовательский интерфейс с обеспечением адаптивности, кроссбраузерности и высокой производительности.

4. Спроектировать серверную архитектуру и базу данных, разработать API и реализовать бизнес-логику обработки ключевых процессов (бронирование, оплата, управление каталогом услуг, формирование отчетности).

5. Провести интеграцию клиентской и серверной частей, обеспечить взаимодействие с внешними сервисами (платежные шлюзы, геоинформационные системы).

6. Выполнить комплексное тестирование платформы (юзабилити-тестирование, функциональное, нагрузочное), оптимизировать производительность и устранить выявленные недостатки.

2.3 Команда проекта

Разработка платформы осуществляется командой студентов Российского технологического университета МИРЭА. Структура команды сформирована с учётом распределения ключевых ролей, обеспечивающих комплексный подход к созданию цифрового продукта. Каждый участник команды несет ответственность за свой функциональный блок, а совместная работа построена на принципах кросс-функционального взаимодействия, что позволяет синхронизировать дизайн, клиентскую и серверную разработку на всех этапах реализации проекта.

2.4 Бизнес-модель по выводу на рынок продукта

Бизнес-модель платформы ориентирована на российский рынок пляжных и сопутствующих сервисов и строится по платформенному принципу: клиент получает единое цифровое окно для выбора, бронирования и оплаты услуг, а партнер - канал продаж, инструменты управления заказами и продвижения. В рамках данного раздела экономическая логика раскрывается концептуально, без детального расчета эффективности.

Основой монетизации выступает сочетание трех инструментов: комиссии с операций, платной подписки для партнеров и промо-размещений внутри платформы. Комиссионная модель является базовой, поскольку напрямую связана с фактическим объемом заказов и позволяет масштабировать выручку по мере роста партнерской сети. Подписка применяется для организаций, которым необходим расширенный набор функций: приоритетное размещение, аналитика заказов, выделение карточки

в поиске и снижение комиссии. Структура бизнес-модели платформы представлена в таблице 2.1.

Таблица 2.1 - Структура бизнес-модели платформы

Элемент	Содержание	Роль для клиента	Роль для партнера
Ценностное предложение	Единая платформа бронирования и заказа услуг	Экономия времени и удобная оплата	Новый канал продаж и цифровой учет
Комиссия	Платеж с фактических заказов	Без отдельной платы за вход	Оплата только при наличии спроса
Подписка	Расширенные функции и сниженная комиссия	Более заметные предложения	Приоритет, аналитика, продвижение
Промо-размещение	Платное повышение видимости карточки	Быстрый доступ к релевантным услугам	Усиление сезонных продаж

Для логики монетизации принят средний дневной чек одного клиента на уровне 3 000 руб. Такой показатель отражает типичный набор услуг в пределах одного дня пребывания на пляжной территории: аренда места, сопутствующий заказ в кафе или доставка, а также дополнительные сервисы. Стандартная комиссия может быть установлена на уровне 11%, а для партнеров с подпиской - 9%. Подписка предлагается по фиксированной цене 5 000 руб. в месяц; ее задача состоит не только в формировании регулярного дохода, но и в удержании партнеров за счет сниженной комиссии и расширенного набора функций. Ключевые параметры монетизации сведены в таблицу 2.2.

Таблица 2.2 - Ключевые параметры монетизации

Показатель	Базовое значение	Назначение	Комментарий
Средний дневной чек клиента	3 000 руб.	База для комиссионной модели	Расходы одного посетителя за день
Стандартная комиссия	11%	Основной доход платформы	Применяется к большинству операций
Комиссия по подписке	9%	Стимулирование удержания партнеров	Действует при активной подписке
Подписка для партнера	5 000 руб./мес.	Дополнительный регулярный доход	Включает приоритет и аналитику

Вывод продукта на рынок должен осуществляться поэтапно. Сначала формируется техническое ядро, затем платформа проходит ограниченный пилот в одном регионе, и только после этого масштабируется партнерская сеть и маркетинговая активность. При этом внедрение начинается до публичного релиза: на стадии интеграций подключаются первые организации,

тестируются реальные сценарии бронирования и отрабатываются операционные процессы. Дорожная карта вывода продукта на рынок представлена на рисунке 2.1, а состав этапов вывода – в таблице 2.3.

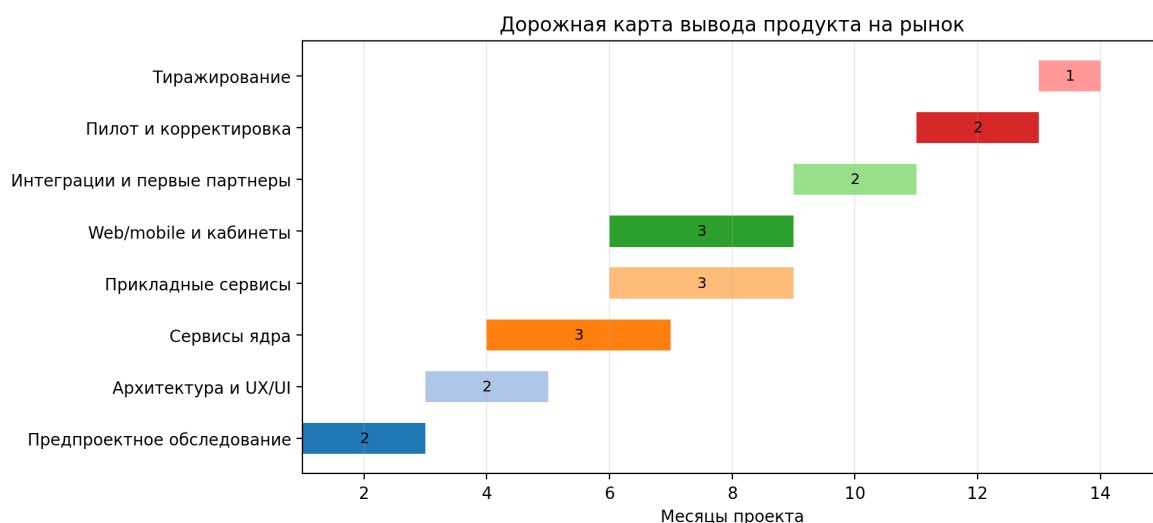


Рисунок 2.1 - Дорожная карта вывода продукта на рынок

Таблица 2.3 - Этапы вывода продукта на рынок

Этап	Срок	Содержание	Результат
Предпроектное обследование	1-2 мес.	Уточнение сценариев, ролей и пилотного региона	Подтвержденный объем внедрения
Разработка ядра и интерфейсов	3-8 мес.	Архитектура, UX/UI, базовые сервисы, web/mobile	Технически готовая версия
Интеграции и пилот	9-12 мес.	Подключение первых партнеров и ограниченный запуск	Проверка бизнес-сценариев
Тиражирование	13-14 мес.	Расширение партнерской сети и маркетинга	Выход на устойчивый рост

2.5 Описание нового продукта

Разрабатываемый продукт представляет собой специализированную цифровую платформу для координации услуг малого и среднего бизнеса в зонах пляжного отдыха. Платформа объединяет клиентов, партнеров и администратора в единой цифровой среде. Клиент получает возможность быстро найти нужную услугу по геолокации, времени и формату сервиса; партнер - управлять размещением, заказами и доступностью; администратор - контролировать контент, обращения и корректность операций.

Новый характер продукта определяется сочетанием нескольких функций в рамках одной платформы. В отличие от разрозненных решений, где бронирование, питание, доставка и коммуникация часто существуют в разных каналах, платформа сводит основные пользовательские сценарии в один интерфейс. Это снижает транзакционные издержки для клиента и повышает управляемость сервиса для партнера. Укрупненная схема платформы представлена на рисунке 2.2, а модульный состав минимально жизнеспособного продукта – в таблице 2.4.

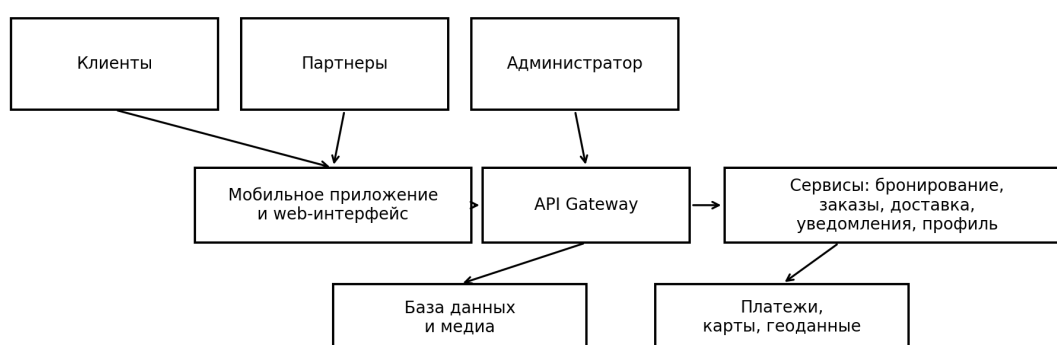


Рисунок 2.2 - Укрупненная схема платформы

Таблица 2.4 - Модульный состав минимально жизнеспособного продукта (MVP)

Модуль	Основная функция	Практическое значение
Аутентификация и роли	Регистрация, вход, разграничение доступа	Работа клиентов, партнеров и администратора
Организации и точки	Карточки заведений, описание услуг, расписание	Цифровое представление партнеров
Бронирование	Выбор времени, места и подтверждение заказа	Базовый сценарий пляжного сервиса
Заказы и доставка	Оформление, изменение и контроль статуса	Интеграция кафе и сопутствующих услуг
Уведомления	Push-, e-mail- и системные сообщения	Снижение числа пропущенных действий
Медиа и контент	Фотографии, баннеры, визуальные материалы	Повышение конверсии карточек
Административный контур	Модерация, справочники, настройки	Управление качеством платформы

К числу ключевых преимуществ платформы относятся единая точка входа для клиента, повышение видимости локальных организаций, управляемая система бронирования, возможность подключения доставки и сопутствующих сервисов, а также адаптация под выраженную сезонность

спроса. За счет этого платформа может использоваться не только в период максимальной пляжной нагрузки, но и в другие месяцы года, когда структура спроса смещается в сторону кафе, семейных сервисов, экскурсий и локального досуга.

2.6 Потенциал продвижения стартап-проекта: SEO-продвижение и маркетинг

Потенциал продвижения проекта связан с тем, что спрос на услуги носит локальный и ситуационный характер: пользователь ищет ближайшее место, доступный формат отдыха и понятную цену. Поэтому на старте основным каналом привлечения выступает локальное SEO-продвижение, дополненное картографическими сервисами, контекстной рекламой, партнерским контентом и прямым продвижением внутри самой площадки.

Для целей настоящей работы сезонная структура спроса рассматривается в двух состояниях: в период активного пляжного спроса и вне пляжного сезона. В период активного пляжного спроса основную долю формируют бронирование мест, аренда инвентаря и водные активности. Вне пляжного сезона спрос на пляжные места становится минимальным, а структура потребления смещается в сторону кафе, доставки и иных локальных сервисов. Изменение структуры спроса в течение года показано на рисунке 2.3, а основные каналы продвижения платформы представлены в таблице 2.5.

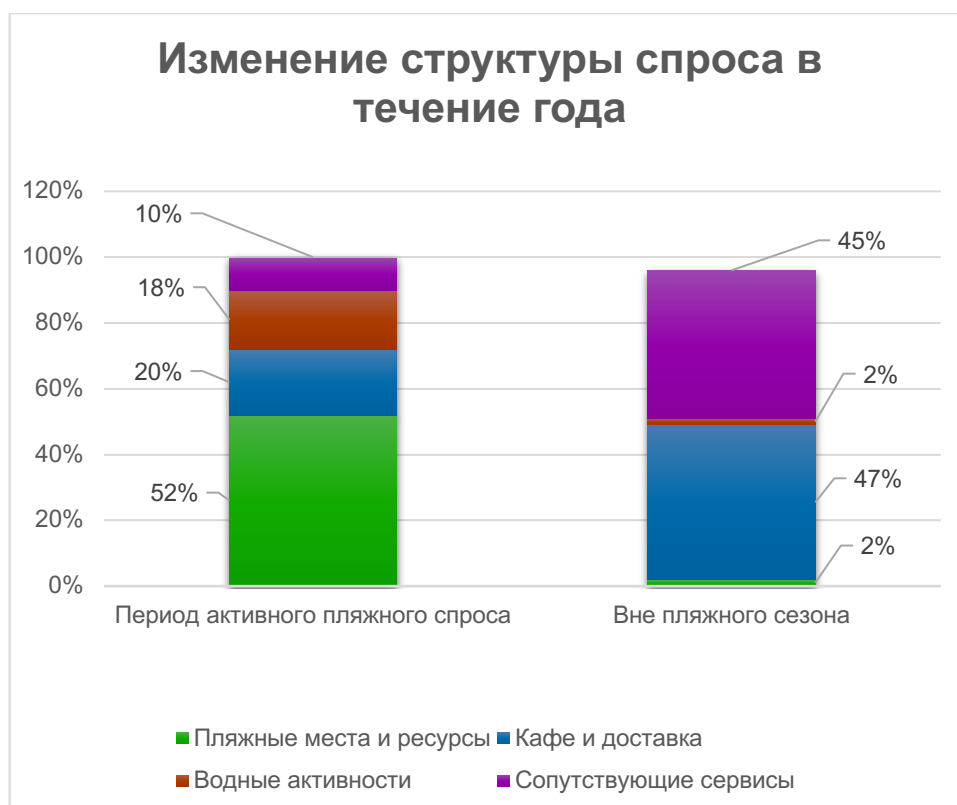


Рисунок 2.3 - Изменение структуры спроса в течение года

Таблица 2.5 - Каналы продвижения платформы

Канал	Назначение	Период максимальной отдачи	Практическая задача
Локальное SEO	Привлечение горячего спроса	В течение всего года	Вывод карточек услуг в поиске по геозапросам
Карты и геосервисы	Поиск рядом с локацией пользователя	Пляжный сезон	Перехват спроса в момент выбора места
Контекстная реклама	Поддержка запуска и пилота	Пляжный сезон	Быстрое привлечение первой аудитории
Социальные сети и контент	Формирование узнаваемости	Весь год	Продвижение сценариев отдыха и акций
Партнерские размещения	Стимулирование заказов и повторных визитов	Вне пляжного сезона	Перенос акцента на кафе и досуг

В период активного пляжного спроса маркетинговый акцент делается на бронирование мест, аренду ресурсов и водные активности. Вне пляжного сезона коммуникация переносится на кафе, доставку и сопутствующие локальные сервисы, которые сохраняют операционную активность партнеров внутри платформы. Такая модель позволяет поддерживать присутствие

сервиса в течение всего года и перераспределять спрос между различными категориями услуг в зависимости от сезона.

Таким образом, потенциал продвижения стартап-проекта определяется сочетанием локального SEO, сезонно адаптированного маркетинга и платформенной модели, в которой продвижение партнеров является встроенной функцией продукта.

2.7 Характеристика результатов индивидуальной работы по backend-разработке

В рамках общего проекта Coastify индивидуальная backend-часть связана с реализацией серверного контура платформы. Она дополняет общую концепцию продукта, бизнес-модель и описание минимально жизнеспособного продукта, рассмотренные выше, и показывает, каким образом проектная идея была перенесена на уровень серверной архитектуры, API, модели данных, интеграций и тестирования.

Результаты backend-разработки рассматриваются по трем взаимосвязанным направлениям: проектирование серверной архитектуры и модели данных, реализация API и интеграционного контура, а также тестирование, обеспечение безопасности и подготовка серверной части к эксплуатации.

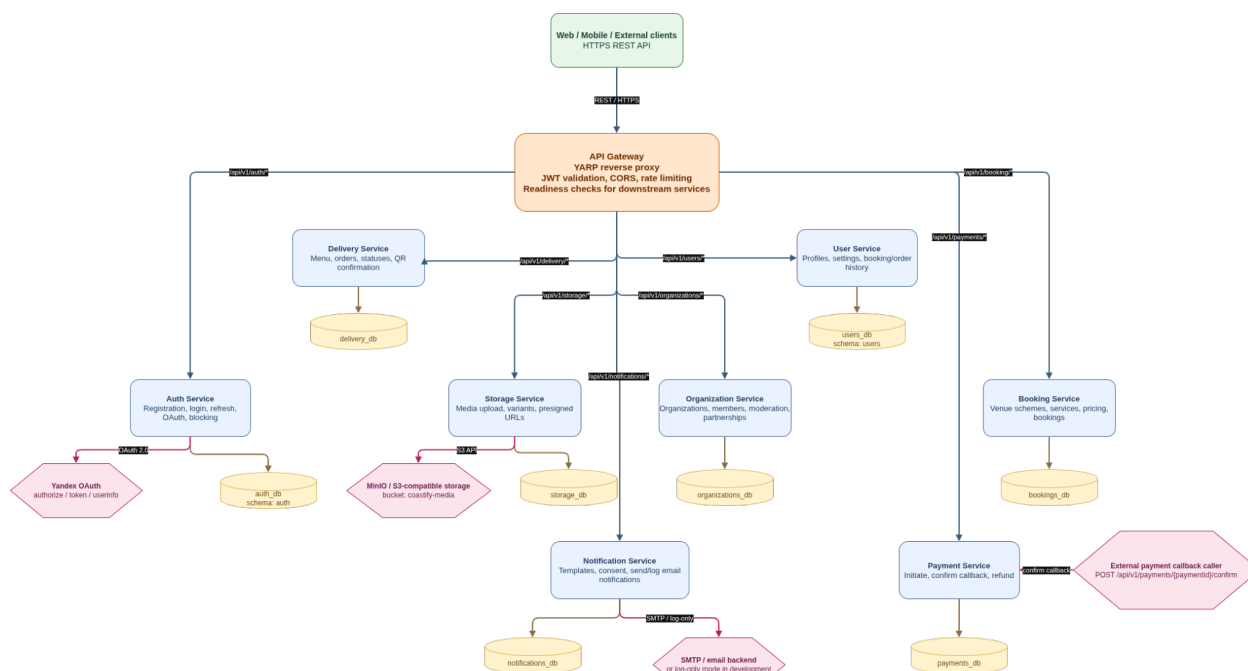


Рисунок 2.4 – Общая схема backend-контура платформы Coastify

2.7.1 Результаты разработки серверной архитектуры и модели данных

Серверная часть платформы реализована на базе микросервисной архитектуры с центральной внешней точкой входа через API Gateway. В составе решения выделены девять сервисов: ApiGateway, AuthService, UserService, OrganizationService, BookingService, DeliveryService, NotificationService, StorageService и PaymentService. Такое разбиение позволило выделить независимые доменные области, закрепить за каждым сервисом собственную ответственность и избежать построения общего перегруженного серверного ядра.

Внутренняя организация большинства сервисов построена по схеме Api – Application – Domain – Infrastructure. Такое разделение отделяет HTTP-слой от прикладных сценариев, доменных сущностей и инфраструктурных адаптеров. На практике это обеспечивает изоляцию бизнес-правил от деталей хранения данных и внешних интеграций, что особенно важно для сервисов OrganizationService, BookingService и DeliveryService.

Одним из ключевых результатов стала реализация модели database-per-service. Для каждого основного домена используется отдельная база данных PostgreSQL: auth_db, users_db, organizations_db, bookings_db, delivery_db, notifications_db, storage_db и payments_db. Такой подход позволяет развести миграции, исключить прямые межсервисные SQL-зависимости и сохранить доменную изоляцию на уровне хранения данных.

Состав backend-сервисов и их прикладная роль представлены в таблице 2.6.

Таблица 2.6 – Состав backend-сервисов и их роль в проекте

Сервис	Назначение	Хранилище	Практический результат
Auth	Регистрация, аутентификация, refresh, reset password, OAuth	PostgreSQL	Сформирован единый контур доступа и управления сессиями
User	Профиль пользователя, настройки, история заказов и бронирований	PostgreSQL	Обеспечена персонализация и агрегация пользовательских данных
Organization	Организации, роли, партнерства, аудит, модерация	PostgreSQL	Реализован управляемый контур работы бизнеса в системе
Booking	Схемы, услуги, pricing, availability, бронирования	PostgreSQL	Реализован основной сценарий бронирования ресурсов и услуг
Delivery	Меню, заказы, статусы, доставка	PostgreSQL	Добавлен сопутствующий сервис заказа и доставки
Notification	Шаблоны, согласия, отправка уведомлений	PostgreSQL	Централизован коммуникационный контур
Storage	Загрузка, хранение и выдача медиа	PostgreSQL + MinIO	Подготовлен отдельный media-сервис

Payment	Инициация и подтверждение платежей, возвраты	PostgreSQL	Сформирован платежный MVP-контур
Gateway	Маршрутизация, edge-security, rate limiting, CORS	–	Обеспечена единая внешняя точка входа

Отдельным результатом backend-разработки стало формализованное описание предметных сущностей и их жизненных циклов. В AuthService реализованы пользователи, внешние логины, refresh tokens, password reset tokens и идемпотентные записи регистрации. В BookingService описаны схемы размещения, сервисы, pricing rules, бронирования и история изменения статусов. В DeliveryService реализованы категории меню, позиции, заказы и история смены статусов. В OrganizationService выделены организации, участники, партнерства, подтверждения отложенных операций и журнал аудита.

Укрупненная схема размещения ключевых сущностей по микросервисам и базам данных представлена на рисунке 2.5. Результаты проектирования и реализации базы данных систематизированы в таблице 2.7.

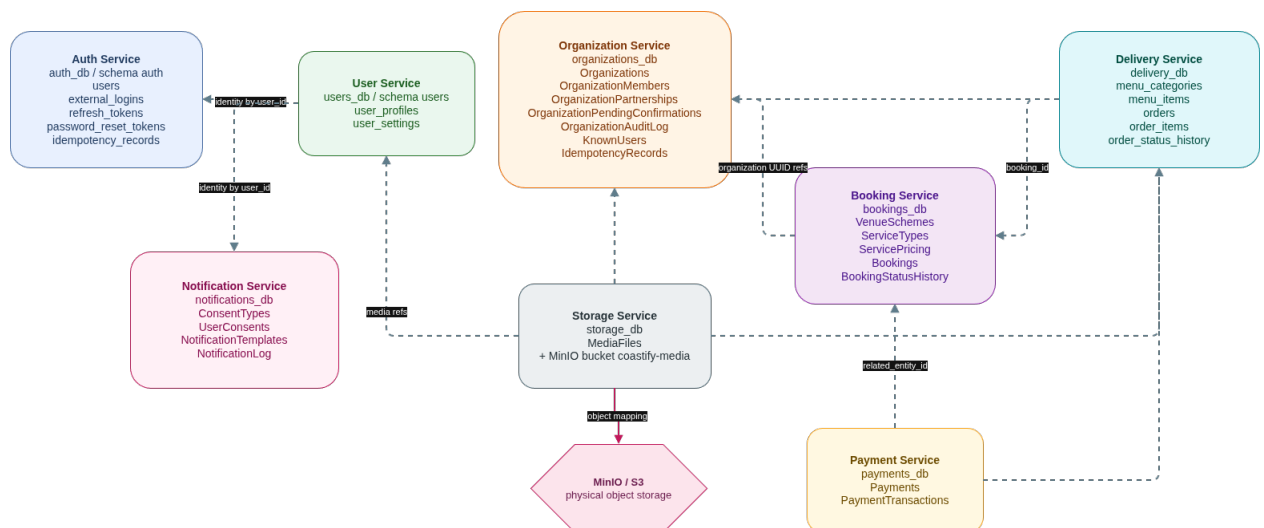


Рисунок 2.5 – Укрупненная схема размещения ключевых сущностей по микросервисам и базам данных

Таблица 2.7 – Ключевые доменные сущности и реализованные бизнес-правила

Сервис	Ключевые сущности	Бизнес-правила и результат
--------	-------------------	----------------------------

Auth	Users, ExternalLogins, RefreshTokens, ResetTokens	Хеширование паролей, rotation refresh token, блокировка; сформирован контур доступа
Org	Organizations, Members, Partnerships, AuditLog	Роли, удаление, модерация; сформирован бизнес-контур
Booking	VenueSchemes, ServiceTypes, ServicePricing, Bookings	Доступность слотов, статусы, QR; реализована обработка бронирований
Delivery	MenuCategories, MenuItems, Orders, StatusHistory	Статусы заказа, отмена, подбор организаций; реализован контур доставки
Notification	ConsentTypes, UserConsents, Templates, Log	Согласия, внутренние и публичные вызовы; централизованы уведомления
Storage	MediaFiles, variants metadata	Тип и размер файлов, resize, presigned URL; реализовано управление медиа

2.7.2 Результаты реализации API, интеграции и защиты пользовательских данных

Следующим направлением индивидуальной работы стала реализация HTTP API и интеграционного контура платформы. Серверная часть построена как API-first система, в которой взаимодействие клиентских приложений и внутренних сервисов осуществляется через согласованный набор HTTP-маршрутов. Практический результат этой части работы состоит в создании единого внешнего контура доступа, согласованной схемы обработки ошибок и воспроизводимого подхода к межсервисному взаимодействию.

Внешней точкой входа выступает ApiGateway на базе YARP. Через gateway реализованы public и protected маршруты, проверка JWT до передачи запроса в downstream-сервисы, CORS, fixed-window rate limiting и security headers. Реализация конфигурации внешнего gateway-контура приведена в приложении А.

В ходе реализации API был выработан общий формат ошибок на основе application/problem+json. В ответах используются correlationId, requestId и, в зависимости от сервиса, traceId. Дополнительно применен fail-fast-подход к конфигурации: критические ошибки обязательных параметров и зависимостей выявляются на этапе старта сервиса, а не в момент пользовательского запроса.

В составе интеграций реализованы OAuth через Яндекс в AuthService, email-уведомления через SMTP или log-only режим в NotificationService, S3-совместимое объектное хранилище MinIO в StorageService и платежный MVP-контур в PaymentService. Внутренние сервисные вызовы строятся преимущественно на синхронном HTTP-взаимодействии через HttpClient. Состав основных внешних и внутренних интеграций представлен в таблице 2.8.

Таблица 2.8 – Реализованные API и интеграционные взаимодействия

Компонент	Интеграция / API	Назначение	Результат
Gateway	YARP Reverse Proxy, JWT validation, public/protected routes	Единая внешняя точка входа	Согласованный edge-контур платформы
Auth	Yandex OAuth, reset password, refresh lifecycle	Внешняя аутентификация и управление доступом	Расширенный контур входа и авторизации
Notification	SMTP / LogOnly	Email-уведомления и согласия	Рабочий контур транзакционных уведомлений
Storage	MinIO / S3-compatible object storage	Хранение изображений и генерация URL	Изолированный media-сервис
Payment	Initiate / confirm callback / refund	Платежный MVP-контур	Подготовлена точка расширения под реальный эквайринг
User	HTTP aggregation из Booking и Delivery	Агрегация истории пользователя	Собрана единая клиентская история
Booking / Delivery / Org	Взаимные HTTP-вызовы	Координация бронирования, доставки и партнерств	Сформировано межсервисное взаимодействие

Отдельный практический результат связан с обеспечением безопасности и защиты пользовательских данных. В контуре аутентификации реализована работа с JWT-токенами и refresh lifecycle, а пароли и служебные токены не хранятся в открытом виде. На уровне gateway используются security headers, rate limiting и разграничение публичных и защищенных маршрутов. Внутри

сервисов доступ к операциям ограничивается ролевой моделью, включающей системные и организационные роли.

С точки зрения прикладной реализации особенно показателен сценарий бронирования, поскольку он сочетает доменные проверки, обращение к модели данных, учет статуса организации, контроль занятости слота и публикацию уведомления. Полная реализация серверного метода создания бронирования приведена в приложении Б.

2.7.3 Результаты тестирования, оптимизации и подготовки серверного контура к эксплуатации

Третьим направлением индивидуальной работы стали тестирование, оптимизация и подготовка backend-контура к эксплуатации. По фактическому состоянию проекта интеграционные тесты реализованы для каждого сервиса отдельно. В тестовый контур входят проверки маршрутизации и безопасности gateway, сценарии регистрации и аутентификации, работа с профилем пользователя, операции над организациями и ролями, сценарии бронирования, оформления заказа, уведомлений, media-операций и платежного MVP.

Тестирование построено не только как локальная проверка отдельных методов, но и как верификация внешнего API-контракта. Для тестового контура использовались изолированные test hosts, адаптеры-заглушки и контейнеризированные сценарии для части сервисов, что позволило проверять поведение API в условиях, близких к реальному запуску. Основные группы тестирования и их результаты представлены в таблице 2.9.

Таблица 2.9 – Основные группы тестирования и их результаты

Объект тестирования	Что проверялось	Инструменты	Результат
Gateway	Public/protected routes, JWT, correlation, readiness	xUnit, integration host	Подтверждена корректная маршрутизация и edge-security
Auth	Register/login/refresh/logout, reset password, OAuth, block/unblock	xUnit, Testcontainers, FluentAssertions	Проверен полный жизненный цикл авторизации

Organization	CRUD, роли, moderation, audit, pending confirmations	xUnit, EF Core test host	Подтверждена корректность ролевой логики
Booking	Availability, booking flows, cancel, status changes, QR	xUnit, fake downstream adapters	Проверены ключевые сценарии бронирования
Delivery	Создание и статус заказа, отмена, notification flow	xUnit, fake adapters	Подтверждена корректная обработка заказа
Storage	Upload, resize, metadata, delete, public aliases	xUnit, fake object storage	Проверена media-логика сервиса
Notification	Consents, template CRUD, internal token, email mode	xUnit	Подтвержден внутренний контур уведомлений
Payment	Initiate, confirm, refund	xUnit	Проверен платежный MVP-сценарий

Показательным примером проверки серверной реализации является интеграционный тест полного сценария бронирования: в рамках одного теста последовательно выполняются создание схемы, создание сервиса, настройка pricing, проверка availability, создание бронирования и получение клиентской истории. Такой тест позволяет проверить связность ключевых API-маршрутов и корректность прикладной логики. Полная реализация сценария приведена в приложении В.

Помимо тестирования были реализованы меры по оптимизации. Для edge-слоя настроен fixed-window rate limiting, ограничивающий избыточный трафик на gateway. Во всех сервисах используются health/live и health/ready endpoints, позволяющие не принимать трафик до установления соединения с базой данных и обязательными downstream-зависимостями. Structured logging через Serilog и корреляционные идентификаторы запросов упростили диагностику межсервисных цепочек вызовов.

В проекте реализован GitLab CI pipeline, включающий стадии restore, build, test, security и package. На этапе restore выполняется восстановление зависимостей решения, на этапе build - сборка сервисов в конфигурации

Release. Этап test включает прогон интеграционных тестов по основным backend-сервисам, что позволяет проверить корректность маршрутизации, авторизации, прикладной логики и межсервисных взаимодействий. На этапе security выполняется статический анализ кода, ориентированный на выявление потенциальных уязвимостей и проблем безопасности. На этапе package формируются publish-артефакты API-проектов, пригодные для дальнейшего развертывания. Контур подготовки backend-сервисов к эксплуатации и автоматизированной сборке показан на рисунке 2.6.

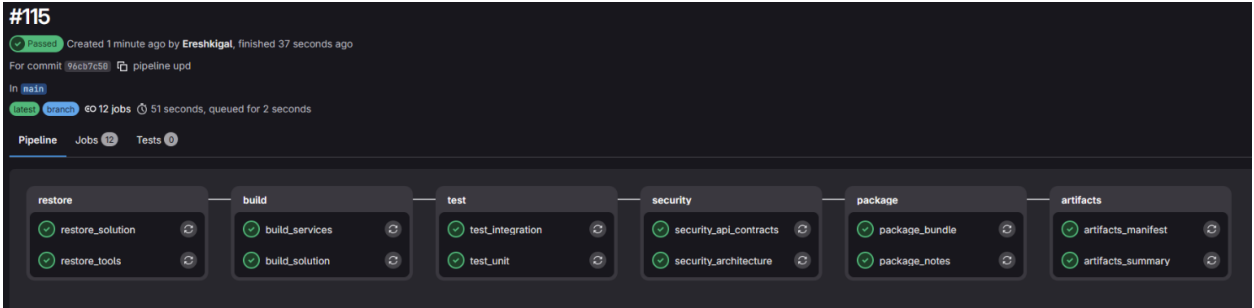


Рисунок 2.6 – Контур подготовки backend-сервисов к эксплуатации и автоматизированной сборке

Итоги работ по оптимизации и подготовке backend-контуров к эксплуатации сведены в таблицу 2.10.

Таблица 2.10 – Результаты оптимизации и подготовки backend-контуров к эксплуатации

Направление	Реализованное решение	Практический эффект	Итог
Logging и трассировка	Serilog, JSON console output, X-Correlation-Id, X-Request-Id, traceparent	Упрощение диагностики в распределенной среде	Повышена наблюдаемость backend-контуров
Health checks	/health/live и /health/ready во всех API	Контролируемый запуск и readiness-проверка	Подготовка к эксплуатации и orchestration
Безопасность edge-слоя	JWT validation, CORS, security headers, rate limiting	Снижение рисков неконтролируемого трафика	Усилен внешний периметр сервиса
CI pipeline	restore, build, test, security, package	Автоматизированная сборка и контроль качества	Получен воспроизводимый контур CI

Публикация артефактов	dotnet publish для API-проектов	Подготовка release-сборок к поставке	Упрощено дальнейшее развертывание
Deployment readiness	Конфигурация через environment variables и готовность health endpoints	Совместимость с дальнейшей контейнеризацией	Сформирована основа для развития контура развертывания

2.8 Выводы по главе

Во второй главе раскрыта разработка проекта Coastify: описана концепция платформы, распределение ролей в команде, бизнес-модель вывода продукта на рынок, состав минимально жизнеспособного продукта и потенциал продвижения. Отдельно рассмотрены результаты индивидуальной backend-работы, связанные с проектированием серверной архитектуры, реализацией API, интеграцией сервисов, защитой пользовательских данных и подготовкой backend-контура к эксплуатации.

Практический результат backend-разработки состоит в создании серверного контура, включающего девять микросервисов, модель хранения данных по принципу database-per-service, PostgreSQL-базы по доменам, MinIO для медиа, набор прикладных HTTP API, механизмы аутентификации и ролевого разграничения доступа. Дополнительно реализованы интеграционные тесты, health-check контур, журналирование, трассировка и GitLab CI pipeline. В совокупности это позволяет рассматривать разработанную серверную часть как инженерную основу платформы, пригодную для дальнейшего развития клиентских контуров и следующего этапа внедрения.

3. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА И ОЦЕНКА ЭФФЕКТИВНОСТИ ПРЕДЛОЖЕННЫХ РЕШЕНИЙ

Экономическое обоснование проекта направлено на определение рыночной состоятельности цифровой платформы агрегации услуг пляжной инфраструктуры, оценку ее финансовой модели и проверку того, может ли продукт обеспечивать устойчивую коммерческую деятельность при поэтапном выводе на рынок. В рамках данного раздела оцениваются параметры рынка, сезонная структура спроса, источники выручки, требуемые инвестиции на запуск, ежемесячная себестоимость функционирования платформы, а также итоговые показатели рентабельности и срока окупаемости.

При построении финансовой модели учитывается, что рынок пляжных и сопутствующих услуг носит ярко выраженный сезонный характер. Это означает, что оценка проекта не может строиться только на пиковых летних месяцах. Для корректного расчета необходимо рассмотреть три режима спроса: высокий сезон, переходный период и период вне пляжного сезона. Такой подход позволяет оценить не только потенциальную доходность продукта, но и его устойчивость в течение календарного года.

3.1 Рыночные предпосылки и экономическая логика продукта

Экономическая состоятельность проекта определяется сочетанием двух факторов. Во-первых, российский рынок цифровых сервисов обладает высокой степенью проникновения: в России насчитывалось около 136 млн пользователей интернета, а уровень интернет-проникновения достигал 94,4% к концу 2025 года [13]. Во-вторых, туристическая и курортная инфраструктура южных регионов страны остается значимым внутренним рынком. По данным Краснодарстата, в 2024 году в коллективных средствах размещения

Краснодарского края было размещено 10,64 млн человек, а доходы отрасли составили 227,6 млрд руб. [14]. Эти показатели подтверждают наличие масштабной пользовательской базы и высокую емкость регионального спроса.

Для рассматриваемого проекта принципиально важно, что цифровая платформа работает не с рынком проживания в целом, а с сопутствующим слоем услуг, которые формируются вокруг пребывания туриста в локации. Сюда относятся бронирование пляжных мест, аренда оборудования, питание, доставка на пляж, локальные активности и дополнительные сервисы. Следовательно, продукт занимает нишу между классическими сервисами размещения и локальными точками обслуживания, обеспечивая цифровую координацию спроса и предложения на последней миле потребления.

С экономической точки зрения проект строится как двухсторонняя платформа. Для клиента ценность состоит в сокращении времени поиска и оформлении услуг в одном цифровом окне. Для партнера ценность выражается в дополнительном канале продаж, управлении заказами и повышении видимости внутри цифровой среды. Следовательно, логика монетизации должна быть напрямую связана с объемом транзакций и одновременно включать стабильный регулярный платеж за расширенные возможности продвижения, аналитики и более выгодные условия работы. Именно поэтому базовая модель выручки сочетает комиссию, подписку и платное продвижение.

Для количественной оценки принят базовый сценарий. Средний дневной чек одного клиента на уровне 3 000 руб. отражает типичный набор расходов в рамках одного посещения пляжной территории: бронирование места или ресурса, заказ в кафе либо доставку, а также одну дополнительную услугу, связанную с пляжной локацией. Такой уровень не является завышенным и может использоваться как расчетная база для определения платформенной комиссии.

3.2 Анализ сезонности спроса и его влияние на выручку

Для целей экономического расчета под высоким сезоном понимается период устойчивого спроса на пляжные места, аренду ресурсов и водные активности, приходящийся преимущественно на июнь–сентябрь. Переходный период включает апрель–май и октябрь, когда интенсивность спроса ниже пиковых значений, но сервисы отдыха, питания и сопутствующих заказов сохраняют востребованность. Период вне пляжного сезона охватывает ноябрь–март и характеризуется снижением спроса на основные пляжные сценарии при сохранении активности кафе, доставки и иных локальных сервисов.

Сезонность не должна рассматриваться исключительно как ограничение. Для платформы она выступает фактором перераспределения структуры спроса. В высокий сезон основную роль играют бронирование мест, аренда ресурсов и водные активности, в переходный период усиливаются смешанные сценарии, а вне пляжного сезона возрастает доля кафе, доставки и сопутствующих локальных сервисов. Дополнительную сглаживающую роль играют партнерская активность внутри платформы и подписочная модель, поддерживающие регулярное присутствие организаций в системе.

В расчетной модели для одной локации принята следующая последовательность спроса по месяцам, клиентских заказов: январь – 900, февраль – 900, март – 1 200, апрель – 2 000, май – 3 200, июнь – 5 200, июль – 6 800, август – 6 800, сентябрь – 5 400, октябрь – 3 200, ноябрь – 1 600, декабрь – 1 100. Такой профиль согласуется с распределением туристического потока в курортной локации и учитывает вклад кафе, доставки и сопутствующих сервисов вне пикового пляжного периода.

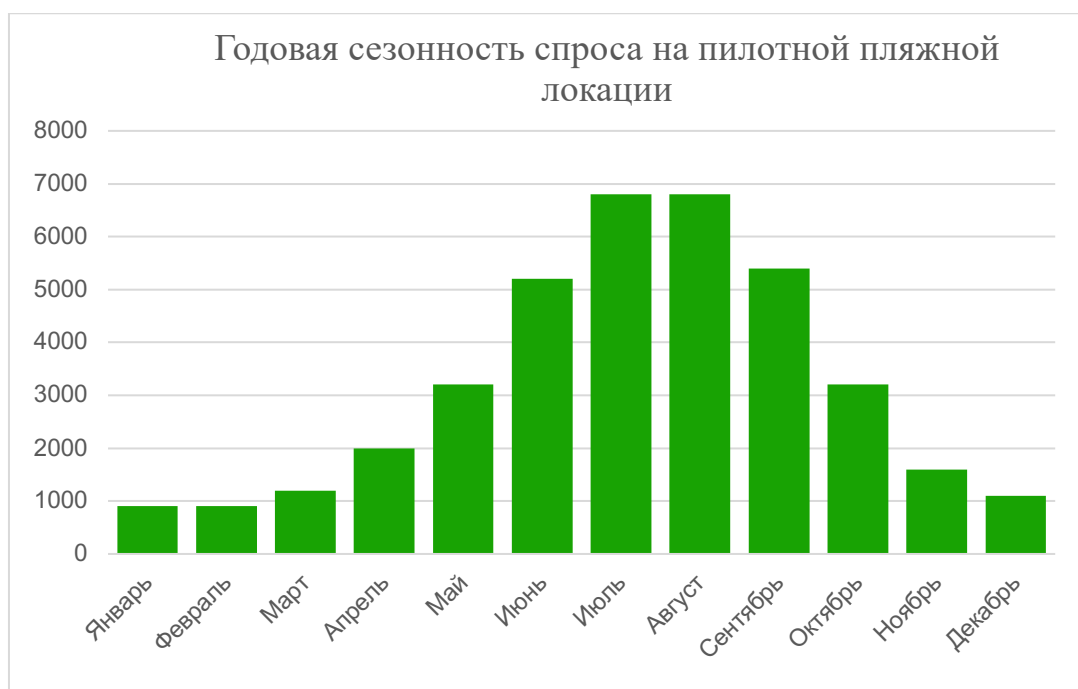


Рисунок 3.1 – Помесячная выручка платформы по сезонным периодам

Таблица 3.1 – Сезонные сценарии спроса и выручки

Период	Партнерская база	Заказы за период	Оборот, руб.	Комиссионный доход, руб.	Подписка и промо, руб.	Выручка за период, руб.
Январь–апрель	42	5 000	15 000 000	1 583 000	255 000	1 838 000
Май–сентябрь	90	27 400	82 200 000	8 592 000	1 071 000	9 663 000
Октябрь–декабрь	58	5 900	17 700 000	1 858 000	263 000	2 121 000

Все последующие финансовые расчеты выполняются именно на основе указанной ежемесячной сезонности, а не через укрупненное деление на высокий и низкий сезон. Годовая сезонность спроса представлена на рисунке 3.1, а расчетные параметры по ключевым периодам года – в таблице 3.1.

С точки зрения годового цикла именно сочетание сезонных режимов делает модель реалистичной. Если бы расчет строился только на пиковых месяцах, рентабельность проекта была бы завышена. Если же ориентироваться только на низкий сезон, экономическая картина была бы излишне консервативной. Принятый трехрежимный подход позволяет увязать экономику платформы с реальной динамикой потребления в курортном регионе.

3.3 Источники выручки и структура бизнес-модели

Выручка платформы формируется из трех источников. Первым и основным источником выступает комиссия с транзакций. Эта составляющая наиболее чувствительна к росту клиентского трафика и потому отражает способность продукта масштабироваться вместе с рынком. Вторым источником является платная подписка партнеров, предоставляющая расширенные возможности продвижения и аналитики, а также более выгодные условия работы в системе. Третьим источником служат промо-размещения и сезонное продвижение внутри платформы, позволяющие монетизировать видимость карточек и специальных предложений.

Комиссионная модель остается доминирующей в структуре доходов, поскольку именно она обеспечивает прямую связь между экономическим результатом платформы и оборотом партнеров. В расчетной модели стандартная ставка комиссии принимается на уровне 11%, а для партнеров с активной подпиской – на уровне 9%. Такая конфигурация позволяет сохранить баланс между доходностью платформы и стимулированием партнеров к более активному участию в экосистеме, тогда как базовые инструменты управления услугами, доступностью ресурсов и заказами остаются частью основной функциональности платформы.

Структура годовой выручки платформы показана на рисунке 3.2. Из диаграммы видно, что доминирующим источником дохода является транзакционная комиссия, тогда как подписка и промо-размещения выполняют стабилизирующую и поддерживающую функции.



Рисунок 3.2 – Структура годовой выручки платформы

Таблица 3.2 – Годовая структура выручки проекта

Источник выручки	Сумма за год, руб.	Доля, %	Экономическая роль
Комиссия с операций	12 033 000	88,2	Основной масштабируемый доход
Подписка партнеров	850 000	6,2	Регулярный партнерский платеж
Промо-размещение и реклама	739 000	5,6	Монетизация видимости и рекламных форматов
Итого	13 622 000	100,0	Суммарная годовая выручка

По данным таблицы 3.2, годовая выручка проекта в расчетном базовом сценарии составляет 13,62 млн руб., из которых 88,2% приходится на комиссию, 6,2% – на подписку и 5,6% – на промо-размещения. Такая пропорция экономически оправдана: большая часть дохода возникает из фактического пользования платформой, а не из фиксированных платежей партнеров.

Для оценки устойчивости выбранной бизнес-модели используются ключевые метрики, отражающие связь между логикой монетизации и фактической динамикой продукта.

Таблица 3.3 – Обязательные ключевые метрики бизнес-модели

Метрика	Базовое значение	Практическое значение
Средний дневной чек клиента	3 000 руб.	База расчета комиссионного дохода
Стандартная комиссия	11 %	Основной уровень платформенной комиссии
Комиссия для партнеров с подпиской	9 %	Стимулирование удержания партнеров
Стоимость подписки	5 000 руб./мес.	Регулярный партнерский платеж
Заказы в самый жаркий месяц	6 800 заказов	Пиковая нагрузка на платформу
Доля партнеров с подпиской	около 28 %	Потенциал регулярной выручки
Средний доход платформы с активного партнера в пик	около 24 000 руб./мес.	Оценка монетизации партнерской базы

3.4 Инвестиции на запуск и ежемесячная себестоимость

Для запуска проекта необходимы стартовые инвестиции, включающие стоимость разработки продукта, первоначальное тестирование, создание базовой инфраструктуры и подготовку пилотного внедрения. В расчетной модели стоимость разработки и стартового запуска проекта принята на уровне 5,4 млн руб. В состав данной суммы включены проектирование и аналитика, backend- и frontend-разработка, UX/UI-подготовка, первичное тестирование, инфраструктурная настройка, административный контур и запуск пилотной локации.

Стартовые инвестиции целесообразно разложить по ключевым компонентам внедрения. Такая детализация позволяет разделить затраты на разработку продукта, инфраструктурную подготовку, пилотный запуск и рекламное продвижение сервиса.

В составе рекламного запуска затраты распределяются между локальной контекстной рекламой, продвижением в картографических сервисах,

таргетированной поддержкой в социальных сетях и стартовым контентным продвижением. В расчетной модели рекламный бюджет запуска принят на уровне 450 тыс. руб., а текущие ежемесячные расходы на продвижение входят в состав операционной себестоимости платформы.

Таблица 3.4 – Структура затрат на реализацию и запуск проекта

Статья затрат	Сумма, руб.	Содержание
Аналитика и проектирование	450 000	Формализация требований, сценариев и архитектурной модели
UX/UI и прототипирование	350 000	Подготовка интерфейсных сценариев и макетов
Backend-разработка	1 800 000	Реализация микросервисов, API, логики и БД
Frontend и клиентская интеграция	950 000	Подготовка клиентского контура и интеграция с backend
Тестирование и обеспечение качества	400 000	Интеграционные проверки, отладка и стабилизация
Инфраструктурная подготовка и пилотное развертывание	1 000 000	Конфигурация окружения, запуск и сопровождение пилота
Рекламный запуск сервиса	450 000	Контекст, карты, таргет и локальное продвижение запуска
Итого	5 400 000	Совокупная стоимость реализации и вывода на рынок

После вывода продукта на рынок платформа переходит в режим ежемесячных эксплуатационных расходов. В отличие от стартовых инвестиций, здесь учитываются не капитальные затраты, а текущая себестоимость функционирования и сопровождения. В рассматриваемом проекте месячная себестоимость составляет 0,95 млн руб. Она формируется из инфраструктурных расходов, сопровождения и развития backend-контуров, поддержки пользователей, онбординга партнеров, маркетинга, административных расходов и амортизации затрат на разработку и внедрение.

Структура ежемесячной себестоимости приведена в таблице 3.5. Принятый уровень затрат соответствует пилотной эксплуатации и учитывает,

что фактическая стоимость сопровождения платформы после запуска будет заметно выше, чем на стадии учебного прототипа.

Таблица 3.5 – Ежемесячная себестоимость функционирования платформы

Статья затрат	Сумма в месяц, руб.	Назначение
Инфраструктура, БД и хранилище	120 000	Серверы, БД, резервирование и медиа-контур
Сопровождение и развитие backend-контуров	220 000	Исправления, доработки, поддержка API
Поддержка пользователей и модерация	130 000	Обработка обращений и контроль контента
Онбординг партнеров и операционные продажи	110 000	Подключение и сопровождение организаций
SEO, контекст, таргет и контент-маркетинг	150 000	Поддержка притока клиентов, партнеров и рекламного запуска
Административные и юридические расходы	80 000	Документооборот, сервисные и правовые расходы
Амортизация разработки и внедрения	140 000	Частичное распределение стартовых вложений
Итого	950 000	Совокупная месячная себестоимость

При месячной себестоимости 0,95 млн руб. минимальный уровень выручки, необходимый для выхода на положительный финансовый результат, должен устойчиво превышать 0,95 млн руб. в месяц. Для достижения уровня рентабельности выше 50% платформа должна увеличивать плотность партнерской сети и объем транзакций по мере перехода от пилотного этапа к масштабированию.

3.5 Финансовые показатели проекта и оценка эффективности

Расчет годовой выручки выполняется на основе помесечной сезонной модели спроса. Годовая выручка проекта в расчетном базовом сценарии составляет 13,62 млн руб., а суммарный оборот услуг, проходящий через платформу, – 114,90 млн руб. При годовой сумме эксплуатационных расходов 11,40 млн руб. проект формирует 2,22 млн руб. годовой прибыли.

Для оценки экономической эффективности важно учитывать не только результат одной пилотной локации, но и эффект масштабирования платформы. При подключении дополнительных пляжных территорий выручка растет за счет увеличения числа заказов и партнеров, тогда как часть затрат остается общей для всей платформы. К таким затратам относятся разработка продукта, часть административного контура, поддержка серверной архитектуры и базовой инфраструктуры. Изменение финансового результата при масштабировании проекта показано на рисунке 3.3.

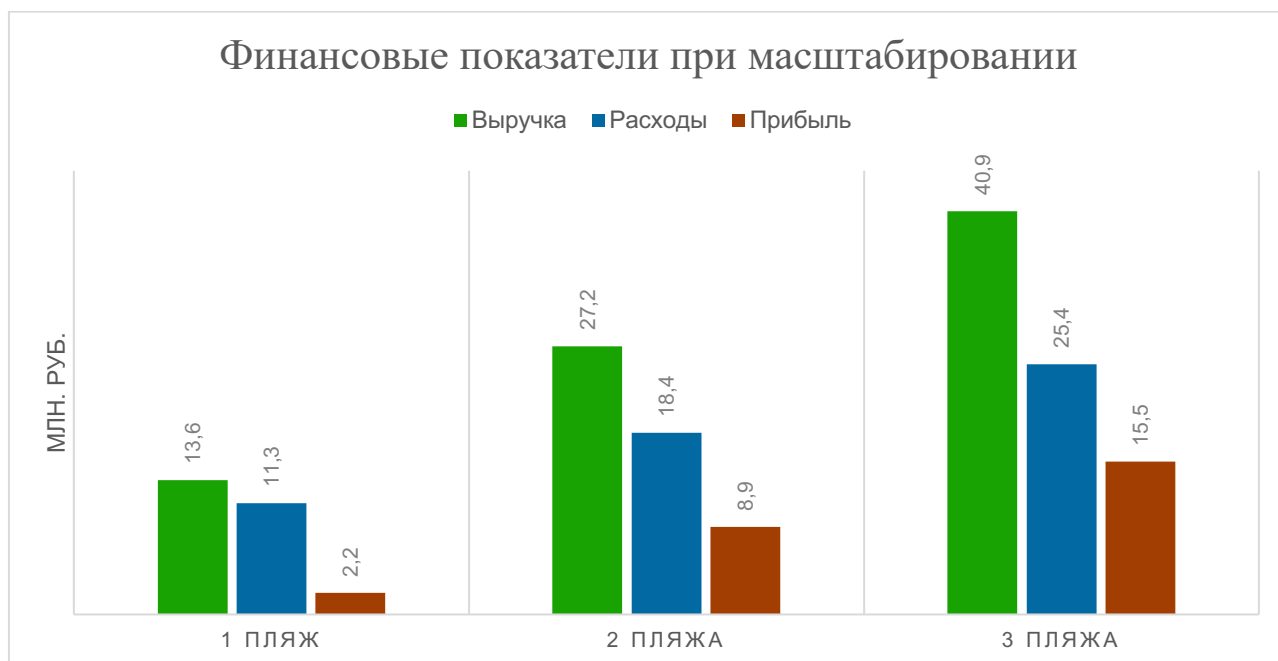


Рисунок 3.3 – Изменение финансового результата при масштабировании проекта

Таблица 3.6 – Ключевые показатели экономической эффективности проекта

Показатель	Значение (1 / 2 / 3 пляжа)	Интерпретация
Годовой оборот услуг через платформу	114,90 / 229,80 / 344,70 млн руб.	Совокупный объем клиентских платежей
Годовая выручка платформы	13,62 / 27,24 / 40,87 млн руб.	Доход от комиссии, подписки и промо
Годовые эксплуатационные расходы	11,40 / 18,39 / 25,38 млн руб.	Расходы растут медленнее выручки за счет общих затрат
Годовая прибыль	2,22 / 8,85 / 15,48 млн руб.	Финансовый результат по сценариям масштабирования

Показатель	Значение (1 / 2 / 3 пляжа)	Интерпретация
Маржа прибыли	16,3% / 32,5% / 37,9%	Отношение прибыли к выручке
Рентабельность затрат	19,5% / 48,1% / 61,0%	Отношение прибыли к эксплуатационным расходам
Стоимость разработки и запуска	5,40 млн руб.	Стартовые инвестиции пилотного запуска
Срок окупаемости стартовых инвестиций	29,2 / 7,3 / 4,2 мес.	Период возврата инвестиций по сценариям

Как видно из таблицы 3.6, экономический эффект проекта существенно усиливается при масштабировании. На одной пляжной локации маржа прибыли составляет 16,3%, а срок окупаемости стартовых инвестиций – около 29 месяцев. При подключении двух пляжей маржа увеличивается до 32,5%, а срок окупаемости сокращается до 7,3 месяца. В сценарии трех пляжей маржа достигает 37,9%, рентабельность затрат – 61,0%, а ориентировочный срок окупаемости составляет около 4,2 месяца.

Маржа прибыли рассчитывается как отношение годовой прибыли к выручке, рентабельность затрат – как отношение годовой прибыли к эксплуатационным расходам, а срок окупаемости – как отношение стартовых инвестиций к годовому финансовому результату. Расчет показывает, что проект в большей степени раскрывает экономический потенциал не на одной пилотной точке, а при тиражировании решения на несколько пляжных локаций.

Экономическая устойчивость проекта обеспечивается за счет сочетания нескольких источников выручки и сохранения партнерской активности в течение календарного года. Это позволяет использовать платформу как стабильный цифровой канал продаж для локального бизнеса внутри пляжной локации.

3.6 Ограничения модели и экономические риски

Несмотря на положительный расчетный результат, финансовая модель проекта содержит ряд ограничений. Ключевое из них состоит в том, что на

одной локации платформа работает в режиме пилотного масштаба и поэтому не в полной мере реализует эффект распределения постоянных затрат.

К экономическим рискам проекта относятся: отклонение фактического среднего чека от базового значения; более медленное подключение партнеров; рост стоимости привлечения клиентов в высокий сезон; недостижение запланированной доли подписки; а также внешние риски курортного рынка, связанные с погодными условиями, транспортной доступностью и общей динамикой внутреннего туризма. Снизить воздействие перечисленных рисков можно за счет поэтапного запуска, работы с пилотным регионом, раннего подключения партнеров, гибкого маркетингового бюджета и сохранения мультисервисной модели спроса вне пляжного сезона.

Отдельного внимания заслуживает риск сезонного снижения транзакционной активности.

Предложенное решение может рассматриваться как экономически приемлемое для пилотного внедрения и последующего масштабирования. На одной пляжной локации проект обеспечивает положительный финансовый результат, а при тиражировании на несколько пляжей улучшаются показатели рентабельности за счет распределения постоянных затрат. Комбинированная модель монетизации, основанная на комиссии, подписке и промо-размещениях, соответствует структуре затрат и ожидаемым результатам проекта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. IEEE Computer Society. SWEBOOK Guide V4.0: Software Architecture, Software Design. 2026.
2. Sandhu R., Ferraiolo D., Kuhn D. The NIST Model for Role-Based Access Control: Towards a Unified Standard. NIST / ACM RBAC, 2000.
3. Azur.ru. Информация о туристическом портале Azur.ru; официальный сайт сервиса отдыха на побережье Краснодарского края, Крыма и Абхазии.
4. MAYAK.TRAVEL. Официальный сайт платформы; разделы о каталоге сервисов, FAQ клиентов и реквизитах оператора.
5. TVIL.RU. Официальные материалы для владельцев объектов; разделы о сервисе, комиссии, календаре занятости и работе с бронированиями.
6. Microsoft Learn. .NET Microservices: Architecture for Containerized .NET Applications.
7. Lewis J., Fowler M. Microservices. MartinFowler.com, 2014; related guidance on decomposition and operational readiness.
8. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine, 2000.
9. OpenAPI Initiative. OpenAPI Specification. Current versions 3.0.4 and 3.1.1.
10. Nottingham M., Wilde E., Dalal S. RFC 9457: Problem Details for HTTP APIs. IETF, 2023.
11. Daraghmi E., Zhang C.-P., Yuan S.-M. Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture. Applied Sciences, 2022.
12. Cockburn A. Hexagonal Architecture (Ports and Adapters). 2005.
13. DataReportal. Digital 2026: The Russian Federation.
14. Краснодарстат. Основные показатели деятельности коллективных средств размещения Краснодарского края за 2024 год.
15. АКИТ. Сводные аналитические данные по рынку интернет-торговли в России.
16. Statcounter Global Stats. Search Engine Market Share in Russian Federation, 2026.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Реализация конфигурации API Gateway

```
using System.Text;
using System.Threading.RateLimiting;
using Coastify.ApiGateway.Middleware;
using Coastify.ApiGateway.Options;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.RateLimiting;
using Microsoft.IdentityModel.Tokens;
using Microsoft.Net.Http.Headers;
using Serilog;
using Serilog.Formatting.Compact;

var builder = WebApplication.CreateBuilder(args);

var jwtOptions =
builder.Configuration.GetSection(JwtOptions.SectionName).Get<JwtOptions>() ??
new JwtOptions();
    jwtOptions.Validate();
var securityMode =
builder.Configuration.GetValue<string>("Security:Mode");

    if (string.IsNullOrEmpty(securityMode) || (securityMode is not
("Strict" or "Trusted")))
    {
        throw new InvalidOperationException("Security:Mode must be
explicitly set to Strict or Trusted.");
    }

    if (builder.Environment.IsProduction() && !string.Equals(securityMode,
"Strict", StringComparison.Ordinal))
    {
        throw new InvalidOperationException("Production environment requires
Security:Mode=Strict.");
    }

var allowedOrigins =
builder.Configuration.GetSection("Cors:AllowedOrigins").Get<string[]>() ??
[];
var allowedMethods =
builder.Configuration.GetSection("Cors:AllowedMethods").Get<string[]>() ??
["GET", "POST", "PUT", "DELETE", "OPTIONS"];
var allowedHeaders =
builder.Configuration.GetSection("Cors:AllowedHeaders").Get<string[]>() ??
["Authorization", "Content-Type", "X-Correlation-Id"];
var permitLimit =
builder.Configuration.GetValue<int?>("RateLimiting:Gateway:PermitLimit") ??
60;
var windowSeconds =
builder.Configuration.GetValue<int?>("RateLimiting:Gateway:WindowSeconds") ??
60;

builder.Host.UseSerilog((context, _, configuration) =>
{
    configuration
        .ReadFrom.Configuration(context.Configuration)
        .Enrich.FromLogContext()
```

```

        .Destructure.ByTransforming<string>(value => value.Replace("\r",
"\r", StringComparison.Ordinal).Replace("\n", "\n",
StringComparison.Ordinal))
        .Enrich.WithProperty("Service", "Coastify.ApiGateway")
        .WriteTo.Console(new RenderedCompactJsonFormatter());
    });

builder.Services.AddSingleton(jwtOptions);
builder.Services.AddProblemDetails();
builder.Services.AddHealthChecks();
builder.Services.AddOpenApi();
builder.Services.AddCors(options =>
{
    options.AddPolicy(
        "default",
        policy =>
        {
            policy.WithMethods(allowedMethods)
                .WithHeaders(allowedHeaders);

            if (allowedOrigins.Length > 0)
            {
                policy.WithOrigins(allowedOrigins)
                    .AllowCredentials();
            }
        });
});

builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.OnRejected = async (context, _) =>
    {
        await ProblemDetailsResponseWriter.WriteAsync(
            context.HttpContext,
            StatusCodes.Status429TooManyRequests,
            "rate_limited",
            "Too many requests",
            "Rate limit exceeded.");
    };

    options.AddPolicy("gateway", context =>
    {
        var partitionKey =
context.Connection.RemoteIpAddress?.ToString() ?? "unknown";
        return RateLimitPartition.GetFixedWindowLimiter(
            partitionKey,
            _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = permitLimit,
                Window = TimeSpan.FromSeconds(windowSeconds),
                QueueLimit = 0
            });
    });
});

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.RequireHttpsMetadata = true;
        options.Events = new JwtBearerEvents
        {
            OnChallenge = async context =>

```



```

        {
            context.HandleResponse();
            context.Response.Headers[HeaderNames.WWWAuthenticate] =
"Bearer";

            await ProblemDetailsResponseWriter.WriteAsync(
                context.HttpContext,
                StatusCodes.Status401Unauthorized,
                "unauthorized",
                "Unauthorized",
                "A valid bearer token is required.");
        },
        OnForbidden = async context =>
        {
            await ProblemDetailsResponseWriter.WriteAsync(
                context.HttpContext,
                StatusCodes.Status403Forbidden,
                "forbidden",
                "Forbidden",
                "The current principal is not allowed to access this
resource.");
        }
    };
    options.TokenValidationParameters = new
TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidIssuer = jwtOptions.Issuer,
        ValidateAudience = true,
        ValidAudience = jwtOptions.Audience,
        ValidateLifetime = true,
        ClockSkew = TimeSpan.Zero,
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtOptions.SigningKey))
    };
});

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("authenticated", policy =>
    {
        policy.RequireAuthenticatedUser();
    });
    policy.AuthenticationSchemes.Add(JwtBearerDefaults.AuthenticationScheme);
});

builder.Services
    .AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

var app = builder.Build();

app.UseMiddleware<ProblemDetailsExceptionHandlerMiddleware>();
app.UseHttpsRedirection();
app.UseCors("default");
app.Use(async (context, next) =>
{
    context.Response.OnStarting(() =>
    {
        if (context.Request.IsHttps)
        {

```

```

context.Response.Headers[HeaderNames.StrictTransportSecurity] = "max-
age=31536000; includeSubDomains";
    }

    context.Response.Headers[HeaderNames.XContentTypeOptions] =
"nosniff";
    context.Response.Headers[HeaderNames.XFrameOptions] = "DENY";
    context.Response.Headers["X-XSS-Protection"] = "1; mode=block";
    context.Response.Headers["Content-Security-Policy"] = "default-
src 'self'";
    return Task.CompletedTask;
});

    await next();
});
app.UseMiddleware<CorrelationHeadersMiddleware>();
app.UseSerilogRequestLogging();
app.UseRateLimiter();
app.UseAuthentication();
app.UseAuthorization();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.MapHealthChecks("/health/live").AllowAnonymous();
app.MapHealthChecks("/health/ready").AllowAnonymous();
app.MapReverseProxy(proxyPipeline =>
{
    proxyPipeline.Use(async (context, next) =>
    {
        context.Response.Headers["X-Gateway-Service"] =
"Coastify.ApiGateway";
        await next();
    });
})
.RequireRateLimiting("gateway");

app.Run();

public partial class Program;

```

ПРИЛОЖЕНИЕ Б

Реализация метода CreateBookingAsync в сервисе бронирования

```
public async Task<Result<BookingCreatedResponse>>
CreateBookingAsync(ActorIdentity actor, CreateBookingRequest request,
CancellationToken cancellationToken)
{
    var validation = await
_createBookingValidator.ValidateAsync(request, cancellationToken);
    if (!validation.IsValid)
    {
        return
ValidationFailure<BookingCreatedResponse>(validation.ToDictionary());
    }

    var service = await
_serviceRepository.FindByIdAsync(request.ServiceId, cancellationToken);
    if (service is null || service.IsDeleted ||
service.OrganizationId != request.OrganizationId)
    {
        return
Result<BookingCreatedResponse>.Failure(ErrorType.BadRequest, new
Error("service_not_found", "Service was not found."));
    }

    var organization = await
_organizationAccessGateway.GetOrganizationAsync(request.OrganizationId,
cancellationToken);
    if (organization is null)
    {
        return
Result<BookingCreatedResponse>.Failure(ErrorType.NotFound, new
Error("organization_not_found", "Organization was not found."));
    }

    if (!string.Equals(organization.Status, "approved",
StringComparison.OrdinalIgnoreCase))
    {
        return
Result<BookingCreatedResponse>.Failure(ErrorType.Conflict, new
Error("organization_not_approved", "Organization is not available for
booking."));
    }

    var timeStart = ParseTime(request.TimeStart);
    var timeEnd = ParseTime(request.TimeEnd);
    if (timeEnd <= timeStart)
    {
        return
Result<BookingCreatedResponse>.Failure(ErrorType.Validation, new
Error("validation_failed", "One or more validation errors occurred.", new
Dictionary<string, string[]>
{
    ["timeEnd"] = ["timeEnd must be greater than
timeStart."]
}
)));
    }

    if (!IsWithinWorkingHours(organization, request.Date, timeStart,
timeEnd))
    {
        return Result<BookingCreatedResponse>.Failure(
```

```

        ErrorType.Unprocessable,
        new Error("organization_closed", "Booking is outside
organization working hours."));
    }

    var sameDateBookings = await
_bookingRepository.ListByDateAsync(request.OrganizationId, request.Date,
cancellationToken);
    if (!string.IsNullOrWhiteSpace(request.SchemaElementId))
    {
        var overlaps = sameDateBookings.Any(x =>
            x.SchemaElementId == request.SchemaElementId &&
            x.Status is not "cancelled" and not
"cancelled_by_organization" &&
            x.TimeStart < timeEnd &&
            timeStart < x.TimeEnd);

        if (overlaps)
        {
            return
Result<BookingCreatedResponse>.Failure(ErrorType.Conflict, new
Error("slot_occupied", "The requested slot is already occupied."));
        }
    }

    var pricingRules = await
_pricingRepository.ListByServiceAsync(service.Id, cancellationToken);
    var totalPrice = BookingPricing.ResolvePrice(request.Date,
timeStart, pricingRules, service.BasePrice);
    var schemaData = service.SchemeId.HasValue ? await
_schemeRepository.FindByIdAsync(service.SchemeId.Value, cancellationToken) :
null;
    var schemaElementLabel = ResolveSchemaElementLabel(schemaData,
request.SchemaElementId);
    var bookingNumber = $"BK-({_clock.UtcNow:yyyyMMdd}-
{Random.Shared.Next(1, 999):000})";
    var booking = Booking.Create(
        Guid.NewGuid(),
        bookingNumber,
        actor.UserId,
        actor.Email,
        actor.Phone,
        service.Id,
        service.OrganizationId,
        request.Date,
        timeStart,
        timeEnd,
        request.SchemaElementId,
        schemaElementLabel,
        totalPrice,
        request.PaymentMethod,
        request.PaymentType,
        request.Comment,
        GenerateQrCode(bookingNumber),
        _clock.UtcNow);

    await _bookingRepository.AddAsync(booking, cancellationToken);
    await _bookingRepository.AppendStatusHistoryAsync(
        BookingStatusHistoryEntry.Create(Guid.NewGuid(), booking.Id,
booking.Status, null, null, booking.CreatedAt),
        cancellationToken);
    await _unitOfWork.SaveChangesAsync(cancellationToken);

```

```

        await
_notificationGateway.PublishBookingCreatedAsync(booking.Id, actor.UserId,
cancellationToken);

        return Result<BookingCreatedResponse>.Success(
            new BookingCreatedResponse(
                booking.Id,
                booking.BookingNumber,
                booking.UserId,
                booking.ServiceId,
                booking.OrganizationId,
                organization.Name,
                service.Name,
                booking.Date,
                booking.TimeStart.ToString("HH:mm",
CultureInfo.InvariantCulture),
                booking.TimeEnd.ToString("HH:mm",
CultureInfo.InvariantCulture),
                booking.SchemaElementId,
                booking.SchemaElementLabel,
                booking.Status,
                booking.TotalPrice,
                booking.PaymentMethod,
                booking.PaymentType,
                booking.PaymentStatus,
                booking.QrCode,
                booking.CreatedAt));
    }

```

ПРИЛОЖЕНИЕ В

Интеграционный тест полного сценария бронирования

```
[Fact]
public async Task FullBookingFlow_Works_ForHappyPath()
{
    await _fixture.ResetDatabaseAsync();
    var client = _fixture.CreateClient();

    var userId = Guid.NewGuid();
    var organizationId = Guid.NewGuid();
    var token = _fixture.CreateAccessToken(userId,
"owner@example.com", "Owner", "User", "+79001234567");

    ConfigureAuthorizedClient(client, token);
    SeedOrganization(organizationId, userId, "owner");

    var schemeResponse = await PostAsJsonAsync(
        client,
        $"/api/v1/booking/organizations/{organizationId}/schemes",
        new CreateVenueSchemeRequest(
            "Main scheme",
            new VenueSchemaDataRequest(
                1000,
                800,
                new SchemaCanvasPoint(500, 0, 90),
                [new SchemaCanvasElement("element_1", "sunbed", 100,
100, 80, 200, 0, "A1")]));

    schemeResponse.StatusCode.Should().Be(HttpStatusCode.Created);
    var schemeId = await ReadGuidAsync(schemeResponse, "schemeId");

    var serviceResponse = await PostAsJsonAsync(
        client,
        $"/api/v1/booking/organizations/{organizationId}/services",
        new CreateBookingServiceRequest("Sunbed", "Beach sunbed",
"with_scheme", schemeId, ["element_1"], 480, 1, 1000m));

    serviceResponse.StatusCode.Should().Be(HttpStatusCode.Created);
    var serviceId = await ReadGuidAsync(serviceResponse,
"serviceId");

    var pricingResponse = await PostAsJsonAsync(
        client,
        $"/api/v1/booking/services/{serviceId}/pricing",
        new UpdateServicePricingRequest([new
PricingRuleRequest(null, "10:00", "18:00", 1500m)]));

    pricingResponse.StatusCode.Should().Be(HttpStatusCode.OK);

    var availability = await
client.GetAsync($"/api/v1/booking/organizations/{organizationId}/availability
?date=2026-06-15&serviceId={serviceId}");
    availability.StatusCode.Should().Be(HttpStatusCode.OK);

    var bookingResponse = await PostAsJsonAsync(
        client,
        "/api/v1/booking/bookings",
        new CreateBookingRequest(serviceId, organizationId, new
DateOnly(2026, 6, 15), "10:00", "18:00", "element_1", "card", "full_prepay",
"Near water"));
```

```
        bookingResponse.StatusCode.Should().Be(HttpStatusCode.Created);
        var bookingId = await ReadGuidAsync(bookingResponse,
"bookingId");

        var details = await
client.GetAsync($"api/v1/booking/bookings/{bookingId}");
        details.StatusCode.Should().Be(HttpStatusCode.OK);

        var userBookings = await
client.GetAsync("api/v1/booking/users/me/bookings?status=active");
        userBookings.StatusCode.Should().Be(HttpStatusCode.OK);
    }
```